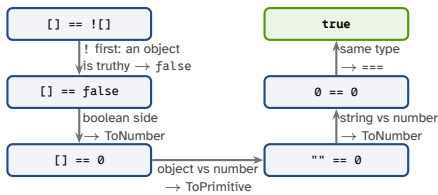


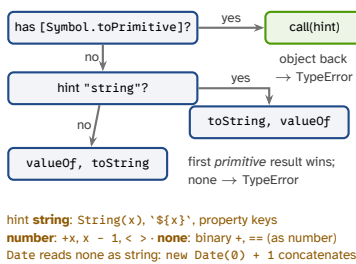
In one line: Seven primitive types (immutable, by value) + **Object** (by identity). Implicit conversions are built from **ToPrimitive**, **ToNumber**, **ToString**, **ToBoolean**, so every “WAT” traces. ES modules **link** the whole graph before any code runs and imports are **live bindings**; require runs a file and returns an object.

1 [] == ![] step by step

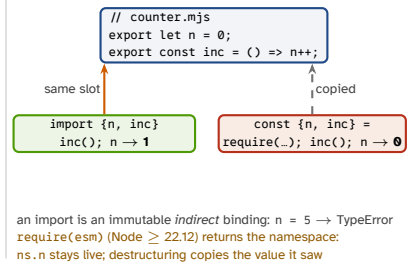


`[]`.valueOf() is the array itself, not a primitive
`→ toString()` → `""`. Same chain makes `true`:
`[0] == false` · `[] == ""` · `"\t\n" == 0` · `[1] == 1`

2 ToPrimitive(obj, hint)



3 live binding vs copy



Types and the four conversions

- Primitives: undefined null boolean number string, symbol (ES2015), bigint (ES2020). All else is an **object**: arrays, functions, wrappers (new Boolean(false) is truthy). “a”.length goes through a temporary wrapper. Assignment and arguments copy the value; for an object that value is a reference — mutating a parameter shows in the caller, reassigning it does not.
- typeof on an undeclared name → “undefined”; on a let/const in its TDZ → `ReferenceError`. Browsers’ document.all (Annex B) is the one object with typeof “undefined”, falsy and == null. Better tests: `x == null` · `Array.isArray` (cross-realm, unlike `instanceof`) · `Object.prototype.toString.call(null)` → “[object Null]” · `Number.isNaN(isNaN("abc"))` is true).

value	typeof	ToBoolean	ToNumber (+x)	ToString ('\${x}')
undefined	"undefined"	false	NaN	"undefined"
null	"object"	false	0	"null"
true	"boolean"	true	1	"true"
-0 · NaN	"number"	false	-0 · NaN	"0" · "NaN"
1e21	"number"	true	1e21	"1e+21"
0n	"bigint"	false	<code>TypeError</code>	"0"
" " · "12n"	"string"	false · true	0 · 12 itself	
"0x1F" · "-0x1F"	"string"	true	31 · NaN	
"1_000" · "1e3"	"string"	true	NaN · 1000	
"Infinity" · "0b11"	"string"	true	Infinity · 3	
Symbol("s")	"symbol"	true	<code>TypeError</code>	<code>TypeError</code>
[] · [7]	"object"	true	0 · 7	" " · "7"
[1,2] · [null]	"object"	true	NaN · 0	"1,2" · ""
{}	"object"	true	NaN	"[object Object]"
() => 1	"function"	true	NaN	its source text
new Date(0)	"object"	true	0	local date text

Falsy: false 0 -0 0n "" null undefined NaN; “0”, “ ”, [], {} are truthy. A string must parse whole after trimming white space; sign only on decimals, no `_`, no n. Arrays join with commas, null/undefined as “”. `ToNumber(10n)` → 10 and `String(Symbol("s"))` → “Symbol(s)” work; `+10n, "" + sym, `${sym}`` throw.

Four equalities

pair	==	===	Object.is	SameValueZero
NaN, NaN	✗	✗	✓	✓
+0, -0	✓	✗	✗	✓
1, "1" · 1n, 1	✓	✗	✗	✗
null, undefined	✓	✗	✗	✗
used by	—	switch, indexOf	Object.is	includes, Map/Set keys

`x == y`, in order: same type → `===` · null/undefined equal each other only · number vs string → `ToNumber(string)` · bigint vs string → `StringToBigInt` (fails → false) · a boolean → `ToNumber` first · object vs primitive → `ToPrimitive`, no hint · bigint vs number → same mathematical value · else false. Not transitive: “0” == 0, 0 == “”, yet “0” == “” is false; null == 0 is false. `< > <= >=`: both sides `ToPrimitive(number)`; two strings compare by UTF-16 code unit (“10” < “9”, “Z” < “a”), else as numbers. `a <= b` is “not b < a”: null > 0 is true, null > 0 false; NaN → false in all four. `3 > 2 > 1` is true > 1: false.

Operator puzzles, traced

expression	result	why
[] + { }	["[object Object]"]	a string operand: + concatenates
{ } + []	0	as a statement: block, + []
[1,2] + [3] · "5" - 2	"1,23" · 3	- is numeric only
[] + null + 1	"null1"	"" + null is a string
[10, 1, 2].sort()	[1, 10, 2]	default sort compares strings
["1", "2", "3"].map(parseInt)	[1, NaN, NaN]	index is the radix
parseInt(0.0000005)	5	String gives “5e-7”
"b"+"a" + +"a" + "a"	"baNaNa"	+"a" is NaN

Seeing the hint: with `o = {[Symbol.toPrimitive]: h => h == "number" ? 42 : h}`, `+o` → 42, `{o}` → “string”, `o + ""` → “default”, `o == "default"` → true.

Numbers and BigInt

- number = IEEE-754 double: integers exact to `MAX_SAFE_INTEGER = 253 - 1 = 9007199254740991`; `2**53 + 1 === 2**53`. `0.1 + 0.2 → 0.30000000000000004`. `Number.EPSILON = 2-52`, the gap after 1 — scale it.
- 0 == 0, but `1/-0 → -Infinity`, `Object.is(-0, 0)` false, `String(-0)` “0”. Math.round ties go to +∞: `2.5 → 3`, `-2.5 → -2`. Bitwise ops use `ToInt32` (>>: `ToUint32`): `2**31 | 0 → -2147483648`; `-1 >>> 0 → 4294967295`.
- BigInt (10n, ES2020): `1n + 1, +1n, Math.max(1n)` throw `TypeError`, but `1n < 1.5` and `1n == 1` work. / truncates (`-7n / 2n → -3n`); `BigInt(1.5)` `RangeError`. `JSON.stringify(1n)` throws; ES2026: `JSON.rawJSON(String(v))` writes it, `reviver{source}` reads it.

??, ||, ?. and defaults

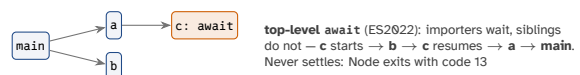
- `a || d` takes d for any falsy a (0, “” lost); `a ?? d` only for null/undefined (ES2020, like ?.). `a ?? b || c` without parentheses is a `SyntaxError`. Parameter and destructuring defaults fire only for undefined: `f(null)` keeps null.
- One `?.` short-circuits the whole chain: `a?.b.c.d`; `(a?.b).c` throws. `a?.b = 1`, new `a?.()` are `SyntaxErrors`. `x ||= d, &&=, ??=` (ES2021) assign only when needed.

Copies — what survives

	{...o}, assign	structuredClone (HTML; Node 17)	JSON round trip
nested object	shared	deep copy	deep copy
cycle	shared	kept	<code>TypeError</code>
Date · Map	shared	copied	ISO string · {}
undefined, NaN	kept	kept	key gone · null
function	shared	<code>DataCloneError</code>	key gone
symbol key	copied	dropped	dropped
BigInt	kept	kept	<code>TypeError</code>
prototype	lost	lost	lost

ES modules — load, link, evaluate

- Load** parses the whole graph; **Link** binds every import to an export — a missing name is a `SyntaxError` before any module of the graph runs; functions are ready, `let/const/class` not; **Evaluate** runs dependencies first, each module once. Always strict; top-level this is undefined.
- `import * as ns: null` prototype, sorted keys, tag “Module”, assigning throws. Cached per URL (Node: new ?query, new instance); browser: deferred, CORS.
- `import(expr)` (ES2020) works in scripts and CommonJS too; a promise of the namespace. **ES2025**: `import c from “./c.json”` with { type: “json” } (default export only; `import(p, { with: {...} })`).
- Cycles**: CJS hands the requirer the *unfinished* exports (missing keys undefined); ESM links fine, but reading a `let/const/class` export before its module ran throws `ReferenceError`; functions already work.



Stage 3: `import defer * as ns` (evaluated on first access), `import source`.

Node.js: CommonJS ↔ ESM (docs v26)

- .mjs ESM · .cjs CJS · .js → nearest package.json “type”, else syntax detection (default since 22.7; slower — set “type”).
- import CJS**: default is *always* module.exports; named imports only where static analysis finds them. Compiled `__esModule` code: the real default sits at `x.default`.
- require(esm)**: the namespace { default; __esModule: true added when a default exists); export { v as “module.exports” } returns v. Unflagged 22.12, stable 25.4; top-level await in its graph → `ERR_REQUIRE_ASYNC_MODULE`.
- ESM: no `require/___dirname` — `import.meta.dirname` (stable 24.0), `createRequire(import.meta.url)`; extensions mandatory; `assert {}` gone (22.0). Separate import/require entry files: **dual package hazard**.